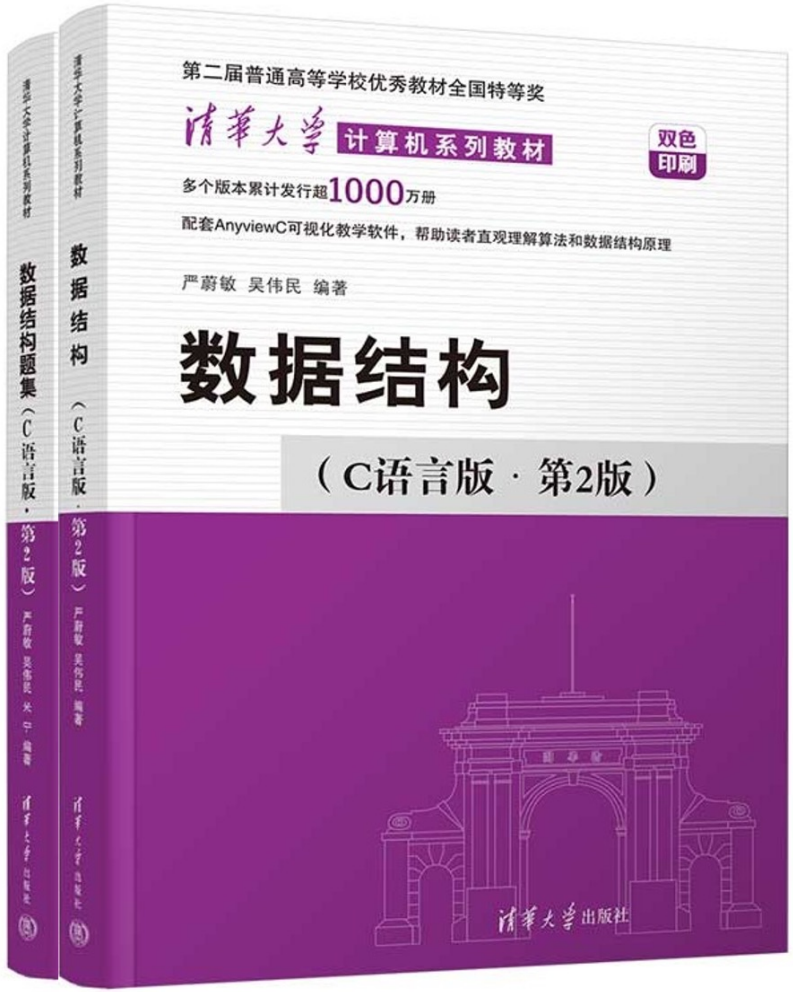


清華大学出版社
TSINGHUA UNIVERSITY PRESS

第8章

动态存储管理

8.1 概 述

动态存储管理的**基本问题**是

1. 系统如何为用户程序（以下统一简称为“用户”）提出的“请求”**分配内存**？
2. 如何回收那些用户不再使用而“**释放**”的内存，以备新的“请求”产生时**重新进行分配**？

称已分配给用户使用的地址连续的内存区为“**占用块**”。

称未曾分配的地址连续的内存区为“可利用空间块”或“**空闲块**”。

例如图8.1(a)所示为依次给8个用户进行**分配**后的系统的内存状态。

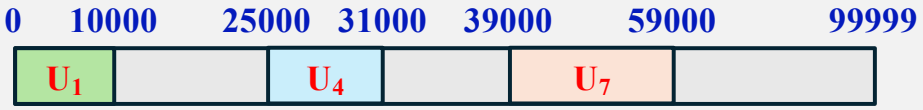


依次**释放**U_5、U_2和U_7后。

问题：如何随时为新的请求分配内存？

策略1：不重新利用释放的空闲块

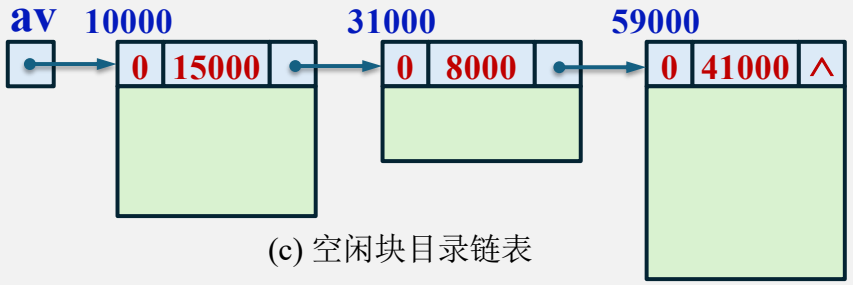
策略2：寻找“合适”的空闲块并分配之，维持空闲块目录表，如图8.2所示



(a) 内存状态

起始地址	内存块大小	使用状况
10000	15000	空闲
31000	8000	空闲
59000	41000	空闲

(b) 空闲块目录顺序表



(c) 空闲块目录链表

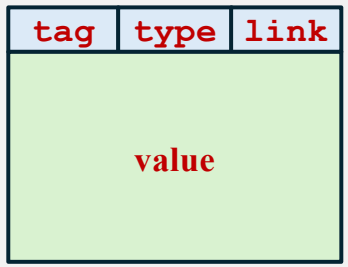
8.2 可利用空间表及分配方法

可利用空间表亦称做**存储池**(Memeray Pool)。根据系统运行的不同情况，可利用空间表可以有下列3种不同的结构形式：

1. 系统运行期间所有用户请求分配的存储量大小相同。通常做法：初始，将可用内存区按所需大小分割成若干大小相同的**块**，然后用指针链接成一个**可利用空间表**。分配时无需查找，只要将第一个结点分配给用户即可；当用户释放内存时，系统只要将释放的空闲块插入在表头即可。这种情况下的可利用空间表实质上是一个**链栈**。是最简单的动态存储管理的方式。

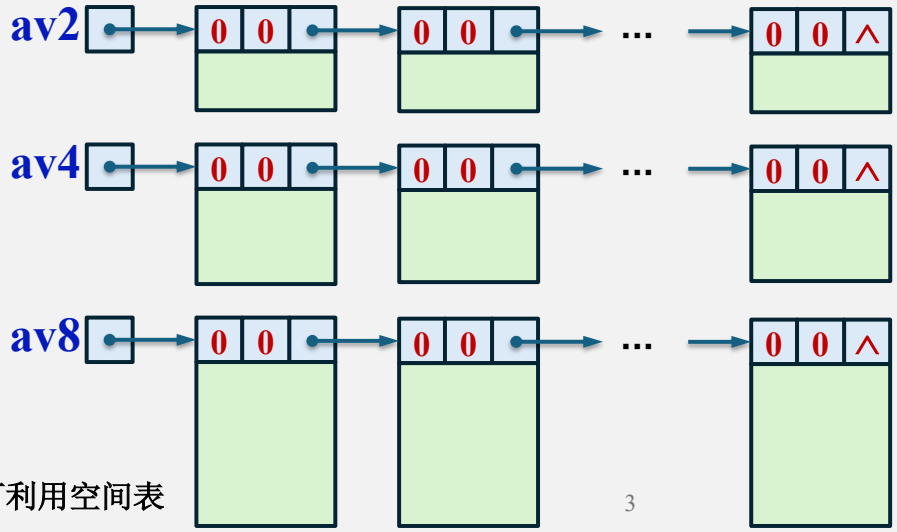
$$\text{tag} = \begin{cases} 0 & \text{空闲块} \\ 1 & \text{占用块} \end{cases}$$

$$\text{type} = \begin{cases} 0 & \text{结点大小为2个字} \\ 1 & \text{结点大小为4个字} \\ 2 & \text{结点大小为8个字} \end{cases}$$



(a) 结点结构

2. 系统运行期间用户请求分配的存储量有若干种大小的规格。建立**若干个可利用空间表**，同一链表中的结点大小相同。**例如**，若用户将请求分配2个字、4个字或8个字的内存块，则相应建立3个结点大小分别为3个字、5个字和9个字的链表，它们的表头指针分别为av2、av4和av8。如图8.3所示，每个结点中的第一个字设有**链域**(link)、**标志域**(tag)和**结点类型域**(type)。

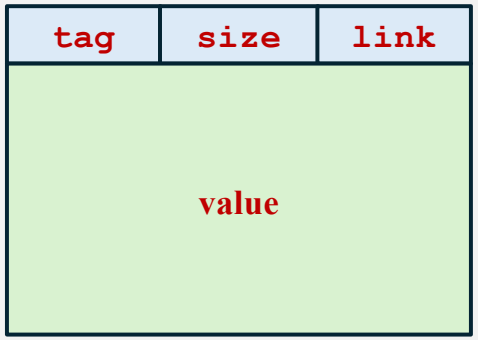


(b) 可利用空间表

3

清華大學出版社 TSINGHUA UNIVERSITY PRESS

3. 系统在运行期间分配给用户的内存块的大小不固定，可以随请求而变。因此，可利用空间表中的结点即空闲块的大小也是随意的。通常，操作系统中的可利用空间表属这种类型。由于链表中结点大小不同，则结点的结构与前两种情况也有所不同，结点中除标志域和链域之外，尚需有一个结点大小域(size)，以指示空闲块的存储量，如图8.4所示。结点中的space域是一个地址连续的内存空间。



$$\text{tag} = \begin{cases} 0 & \text{空闲块} \\ 1 & \text{占用块} \end{cases}$$

图8.4 空闲块的大小随意的结点结构

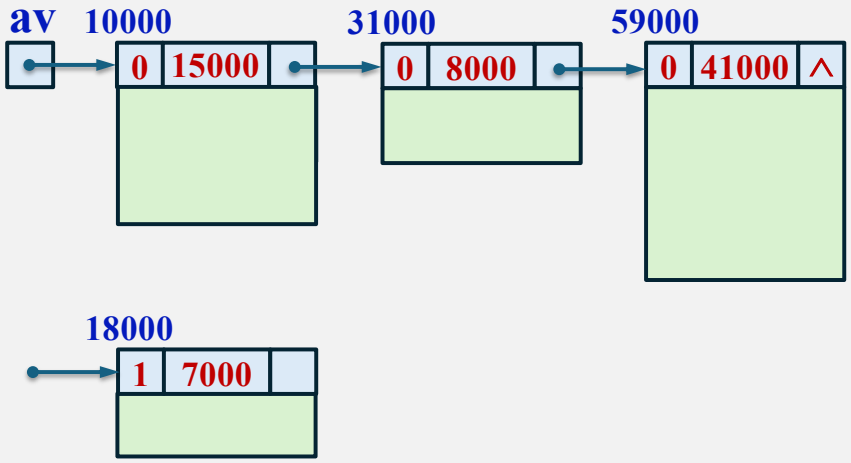
清華大學出版社 TSINGHUA UNIVERSITY PRESS

若可利用空间表中有若干个不小于n的空闲块时，可有3种不同的分配策略：

(1) 首次拟合法。从表头指针开始查找可利用空间表，将找到的第一个大小不小于n的空闲块的一部分分配给用户。可利用空间表本身既不按结点的初始地址有序，也不按结点的大小有序。则在回收时，只要将释放的空闲块插入在链表的表头即可。

例如，在图8.2(c)的状态时有用户U_9进入系统并申请7000B的内存。

系统在可利用空间表中进行查询，发现第一个空闲块即满足要求，则将此块中大小为7000B的一部分分配之，剩余8000B的空闲块仍在链表中，如图8.5(a)所示。图8.5(d)为分配给用户的占用块。

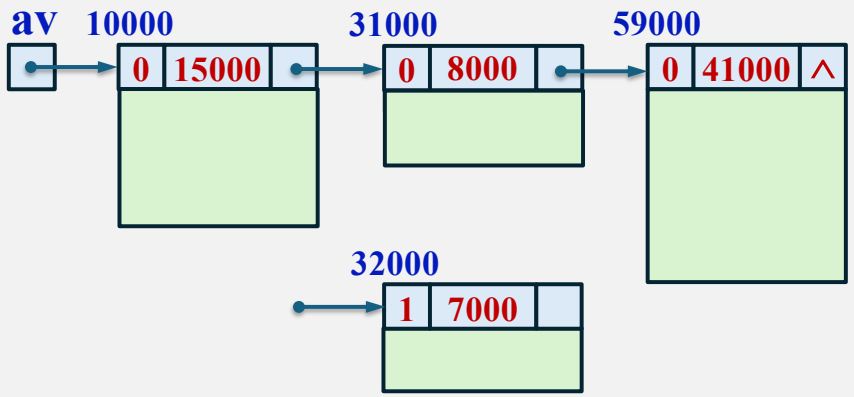


清華大學出版社 TSINGHUA UNIVERSITY PRESS

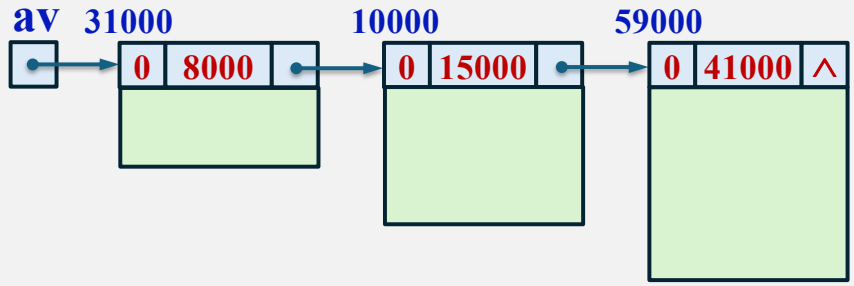
(2) **最佳拟合法**。将可利用空间表中一个不小于n且最接近n的空闲块的一部分分配给用户。则系统在分配前首先要对可利用空间表从头到尾扫视一遍，然后从中找出一块不小于n且最接近n的空闲块进行分配。

显然，在图8.2(c)的状态时，系统就应该将第二个空闲块的一部分分配给用户U_9，分配后的可利用空间表如图8.5(b)所示。

在用最佳拟合法进行分配时，为了避免每次分配都要扫视整个链表。通常，预先设定可利用空间表的结构按空间块的大小自小至大**有序**。由此，只需找到第一块大于n的空闲块即可进行分配，但在回收时，必须将释放的空闲块插入到合适的位置上去。



按块大小升序：

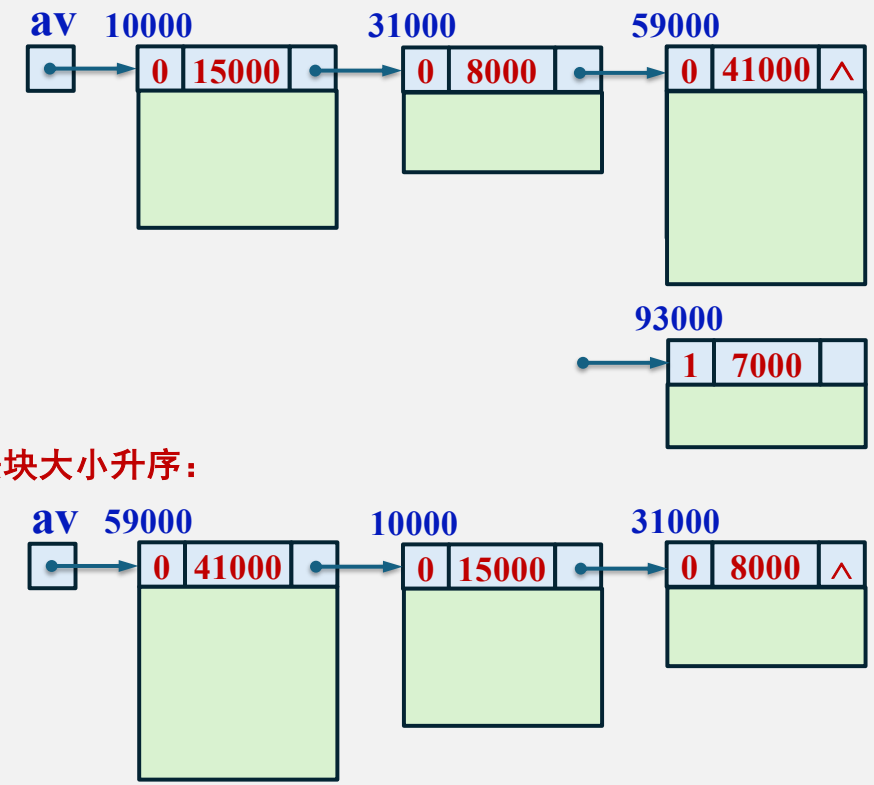


清華大學出版社 TSINGHUA UNIVERSITY PRESS

(3) **最差拟合法**。将可利用空间表中不小于n且是链表中最大的空闲块的一部分分配给用户。

例如在图8.2(c)的状态时，就应将大小为41KB的空闲块中的一部分分配给用户，分配后的可利用空间表如图8.5(c)所示。

显然，为了节省时间，此时的可利用空间表的结构应按空闲块的大小自大至小有序。这样，每次分配无需查找，只需从链表中删除第一个结点，并将其中一部分分配给用户，而剩余部分作为一个新的结点插入到可利用空间表的适当位置上去。当然，在回收时亦需将释放的空闲块插入到链表的适当位置上去。



8.3 边界标识法

边界标识法(boundary tag method)是操作系统中用以进行动态分区分配的一种存储管理方法，它属于上一小节讨论中的第三种情况。

系统将所有的空闲块链接在一个**双重循环链表**结构的可利用空间表中；分配可按首次拟合进行，也可按最佳拟合进行。

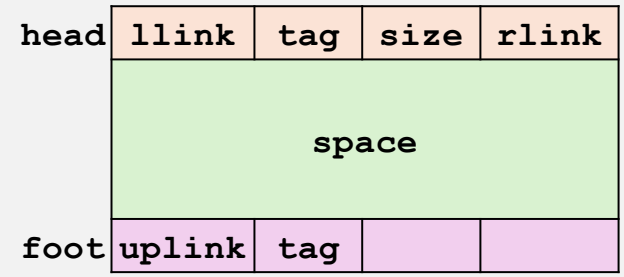
系统**特点**：在每个内存区的头部和底部两个边界上分别设有标识，以标识该区域为占用块或空闲块，使得在回收用户释放的空闲块时易于判别在物理位置上与其相邻的内存区域是否为空闲块，以便将所有地址连续的空闲存储区组合成一个尽可能大的空闲块。

1. 可利用空间表的结构及初始化

可利用空间表中的结点结构如右所示：

可利用空间表的结点结构定义为如下说明的数据类型：

```
typedef struct WORD {    // WORD：内存字类型
  union {               //head和foot分别是结点的第一个字和最后的字
    WORD *llink;        //头部域，指向前驱结点
    WORD *uplink;       //底部域，指向本结点头部
  };
  int  tag;             //块标志，0：空闲，1：占用，头部和尾部均有
  int  size;            //头部域，块大小
  WORD *rlink;          //头部域，指向后继结点
  OtherType  other;     //字的其他部分（用于占位，无须赋值）
} WORD, *Space;         //*Space：可利用空间指针类型
#define FootLoc(p) p+p->size-1 //指向p所指结点的底部
```

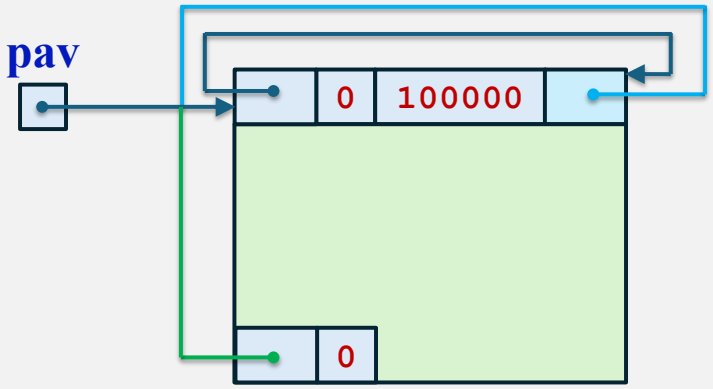


8.3 边界标识法（续）

清華大學出版社 TSINGHUA UNIVERSITY PRESS

1. 可利用空间表的结构及初始化（续）

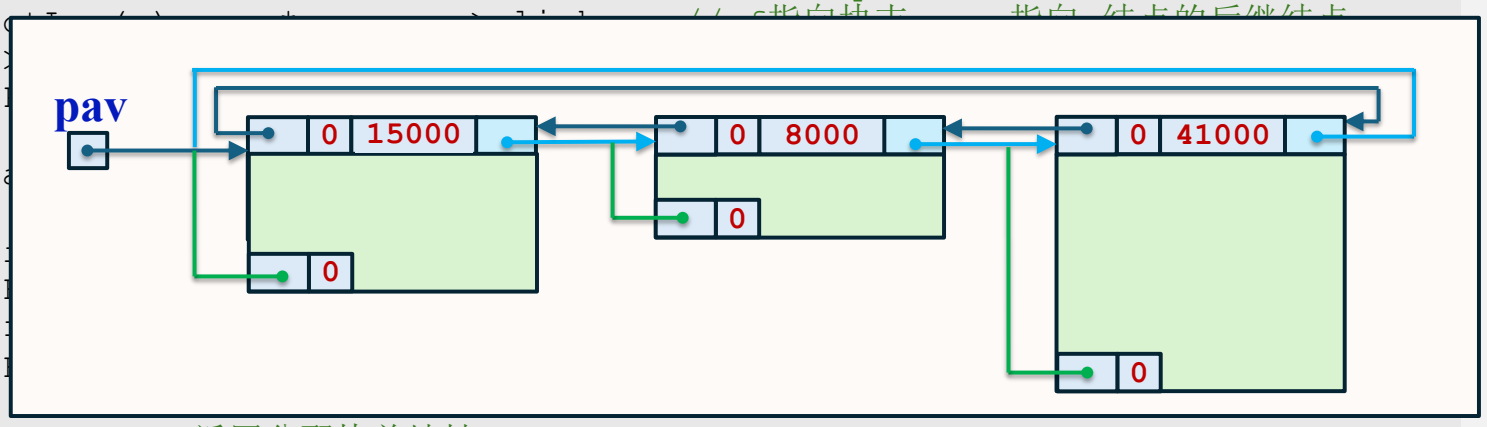
图8.6(a)是一个占有约100KB内存空间的初始可利用空间表。




```
Space InitSpace(int n) {   // 算法 8.1 初始化容量为n的可利用空间表（内存池）
   Space node;
   if (!(node = (WORD*)calloc(n, sizeof(WORD))))   // 分配含n+4个WORD的一块（结点）
      exit(OVERFLOW);
   node[0].size = n-2;      // 实际容量（不含首末两“控制”字）
   node[0].tag  = 0;        // 块首的tag 0: 空闲;  1: 占用
   node[0].llink = node[0].rlink = node;            // 初始内存池为单结点双向循环链表
   node[n-1].tag = 0;    node[n-1].uplink = node;   // 块末指向块首
   return node;                                     // 返回内存池指针
}
```

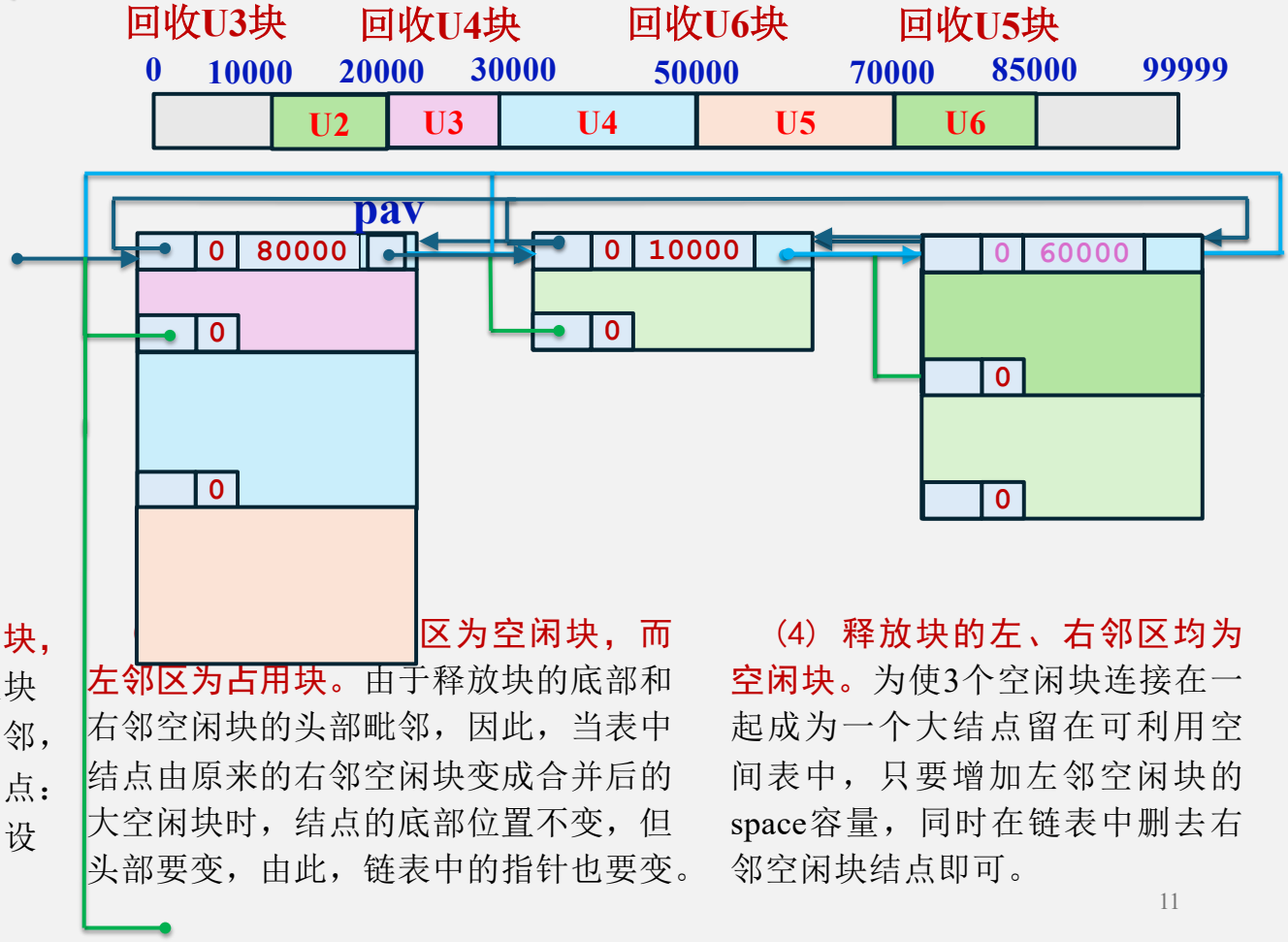
算法 8.2 首次拟合分配

```
#define  e  4     // 约定空闲块容量>e
Space AllocBoundTag (Space *pav, int n) {   // pav的实参应是可利用空间表头指针的地址
  // 若有不小于n+2的空闲块，则分配相应的存储块，并返回其首地址；否则返回NULL。
  // 若分配后可利用空间表不空，则pav指向表中刚分配过的结点的后继结点
  Space p, f, Pa;   int  size = n+2;
  for (p = Pa = *pav;
       p && p->size<size && p->rlink!=Pa; // 查找不小于n+2的空闲块
       p = p->rlink);
  if (!p || p->size<size) return NULL;     // 若找不到，返回空指针
  else {                                   // 否则，p指向找到的空闲块
    f = Fo
    if (p-
      if (
      else
      p->ta
    } else
      p->si
      f =
      p =
      f =
    }
    return p;    // 返回分配块首地址
  }
}
```



算法 8.3 回收 —— 分4种情况分别描述操作要点

清華大學出版社 TSINGHUA UNIVERSITY PRESS



（1）释放块的左、右邻区均为占用块。简单插入即可。

边界标识法在按首次拟合进行分配时对可利用空间表的结构没有任何要求，新的空闲块插入在表中任何位置均可。

简单的做法就是插入在pav指针所指结点之前(或之后)。

（2）释放块的左邻区为空闲块，而右邻区为占用块。由于释放块的头部和左邻空闲块的底部毗邻，因此只要改变左邻空闲块的结点：增加结点的size域的值且重新设置结点的底部即可。

区为空闲块，而左邻区为占用块。由于释放块的底部和右邻空闲块的头部毗邻，因此，当表中结点由原来的右邻空闲块变成合并后的大空闲块时，结点的底部位置不变，但头部要变，由此，链表中的指针也要变。

（4）释放块的左、右邻区均为空闲块。为使3个空闲块连接在一起成为一个大结点留在可利用空间表中，只要增加左邻空闲块的space容量，同时在链表中删去右邻空闲块结点即可。

8.4 伙伴系统

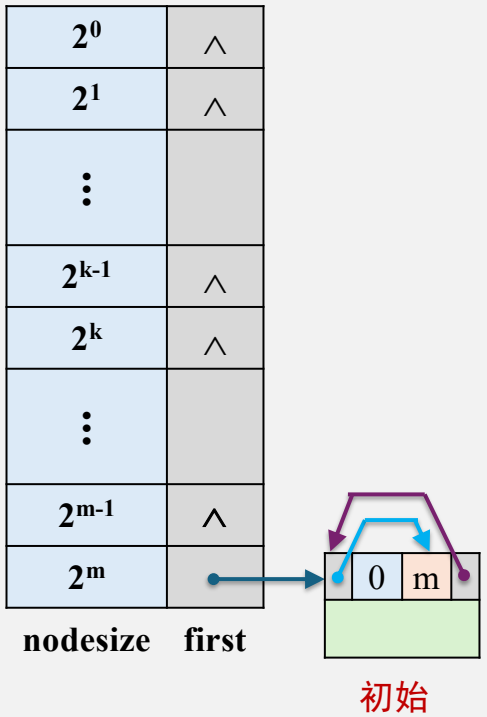
伙伴系统(buddy system)是操作系统中用到的另一种动态存储管理方法。

和边界标识法类似，在用户提出申请时，分配一块大小“恰当”的内存区给用户；在用户释放内存区时即回收。

所不同的是：在伙伴系统中，无论是占用块或空闲块，其大小均为2的k次幂(k为某个正整数)。

例如；当用户申请n个字的内存区时，分配的占用块大小为2^k个字($2^{k-1}<n\leqslant 2^k$)。

可利用空间表中的空闲块大小也只能是2的k次幂。若总的可利用内存容量为2^m个字，则空闲块的大小只可能为2^0、2^1、…、2^m。下面分3个问题展开讨论。



1. 可利用空间表的结构

假设系统的可利用内存空间容量为2^m个字(地址从0到2^m-1)，则在开始运行时，整个内存区是一个大小为2^m的空闲块，在运行了一段时间之后，被分隔成若干占用块和空闲块。为了再分配时查找方便起见，我们将所有大小相同的空闲块建于一张子表中。每个子表是一个双向循环链表，这样的链表可能有m+1个，将这m+1个表头指针用向量结构组织成一个表，这就是伙伴系统中的可利用空间表。

清華大學出版社 TSINGHUA UNIVERSITY PRESS

2. 分配算法

当用户提出大小为n的内存请求时，首先在可利用表上寻找结点大小与n相匹配的子表，若此子表非空，则将子表中任意一个结点分配之即可；若此子表为空，则需从结点更大的非空子表中去查找，直至找到一个空闲块，则将其中一部分分配给用户，而将剩余部分插入相应的子表中。

例如，分配一个2^3块

接着，分配一个2^1块

